



PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Bacharelado em Engenharia da Computação

Marcus Prado Silva

Pedro Henrique Vilaça Valverde

Gabriel El Dine Breguez da Cunha

ESTUDO COMPARATIVO DE IMPLEMENTAÇÕES DO ALGORITMO QUICKSORT

Belo Horizonte

2026

Marcus Prado Silva
Pedro Henrique Vilaça Valverde
Gabriel El Dine Breguez da Cunha

ESTUDO COMPARATIVO DE IMPLEMENTAÇÕES DO ALGORITMO QUICKSORT

Trabalho Prático da disciplina de Projeto e
Análise de Algoritmos, apresentado ao Ins-
tituto de Ciências Exatas e Informática da
Pontifícia Universidade Católica de Minas
Gerais.

Professor: Walisson Ferreira de Carvalho

Belo Horizonte

2026

RESUMO

Este trabalho apresenta um estudo comparativo experimental de três implementações do algoritmo Quicksort: a versão recursiva clássica, uma versão híbrida em que a partição é interrompida para subvetores com menos de M elementos, os quais são ordenados por Insertion Sort, e uma versão híbrida com escolha de pivô por mediana-de-três. As implementações foram desenvolvidas em C++17 sobre um motor único parametrizado, de modo a garantir que as três variantes contabilizem comparações e trocas de forma idêntica. Os algoritmos foram avaliados sobre cinco massas de teste — aleatória, ordenada, inversamente ordenada, com muitos elementos repetidos e quase ordenada — em quatro tamanhos de entrada, entre mil e quinhentos mil elementos, medindo-se tempo de execução, número de comparações e número de trocas. A calibração empírica mostrou que o mínimo do número de comparações é estável em $M \approx 20\text{--}25$, enquanto o mínimo de tempo é ruidoso; adotou-se $M = 40$ pelo critério do menor valor dentro de 2% do melhor tempo. Esse valor é superior à faixa de 5 a 25 recomendada pela literatura clássica, diferença atribuída às características do hardware atual. As versões híbridas reduziram o tempo de execução entre 16% e 45% em relação à versão recursiva pura. Um experimento específico forçou o pior caso do algoritmo, combinando vetor ordenado com escolha inadequada de pivô, e confirmou numericamente a previsão teórica: a razão entre o número de comparações e n^2 convergiu para exatamente 0,5000, com diferença de aproximadamente duas mil vezes em relação à versão com mediana-de-três. A análise crítica revelou que, em dados aleatórios, a versão híbrida executa mais comparações que a versão recursiva e ainda assim é mais rápida, evidenciando que a contagem de operações elementares deixou de ser preditor confiável do tempo de execução em hardware moderno.

Palavras-chave: Quicksort, Insertion Sort, Algoritmos de Ordenação, Análise de Complexidade, Análise Experimental de Algoritmos, Mediana-de-três.

ABSTRACT

This work presents an experimental comparative study of three implementations of the Quicksort algorithm: the classical recursive version, a hybrid version in which partitioning stops for subarrays with fewer than M elements, which are then sorted by Insertion Sort, and a hybrid version using median-of-three pivot selection. The implementations were developed in C++17 on top of a single parameterised engine, ensuring that all three variants count comparisons and swaps identically. The algorithms were evaluated on five test datasets — random, sorted, reverse sorted, with many repeated elements, and nearly sorted — across four input sizes ranging from one thousand to five hundred thousand elements, measuring execution time, comparison count and swap count. Empirical calibration indicated $M \approx 40$ as the optimal value, higher than the range of 5 to 25 recommended by the classical literature, a difference attributed to modern hardware characteristics. The hybrid versions reduced execution time by 16% to 45% compared to the pure recursive version. A dedicated experiment forced the algorithm’s worst case by combining a sorted array with an inadequate pivot choice, numerically confirming the theoretical prediction: the ratio between comparison count and n^2 converged to exactly 0.5000, a difference of approximately two thousand times relative to the median-of-three version. The critical analysis revealed that, on random data, the hybrid version performs more comparisons than the recursive version yet remains faster, showing that counting elementary operations is no longer a reliable predictor of execution time on modern hardware.

Keywords: Quicksort, Insertion Sort, Sorting Algorithms, Complexity Analysis, Experimental Algorithmics, Median-of-three.

LISTA DE FIGURAS

FIGURA 1 – Tempo mediano de execução em função de M , para três tamanhos de entrada.	13
FIGURA 2 – Número médio de comparações em função de M	13
FIGURA 3 – Tempo mediano de execução das três versões, por massa de teste.	15
FIGURA 4 – Número médio de comparações das três versões, por massa de teste. ...	15
FIGURA 5 – Número médio de trocas (movimentações) das três versões, por massa de teste.	15
FIGURA 6 – Redução do tempo de execução das versões híbridas em relação ao Quicksort recursivo.	18
FIGURA 7 – Efeito isolado da mediana-de-três, ambos com $M = 40$: redução percentual das comparações e do tempo em relação ao híbrido simples. Valores positivos indicam vantagem da mediana-de-três.	19
FIGURA 8 – Pior caso: número de comparações e razão comparações/ n^2	21

LISTA DE TABELAS

TABELA 1 – Parametrização das três versões a partir do motor único.	8
TABELA 2 – Massas de teste utilizadas.	10
TABELA 3 – Valores de M obtidos pela calibração empírica. A coluna “ M recomendado” usa o critério do menor M dentro de 2% do melhor tempo; a coluna “ M por comparações” usa o mínimo do número de comparações. ..	14
TABELA 4 – Resultados para a massa Aleatório.	16
TABELA 5 – Resultados para a massa Ordenado.	16
TABELA 6 – Resultados para a massa Inverso.	17
TABELA 7 – Resultados para a massa Repetidos (10 chaves distintas).	17
TABELA 8 – Resultados para a massa Quase ordenado.	18
TABELA 9 – Redução do número de comparações da mediana-de-três em relação ao híbrido simples, ambos com $M = 40$	19
TABELA 10 – Pior caso forçado: vetor ordenado com pivô igual ao primeiro elemento.	20

SUMÁRIO

1	INTRODUÇÃO.....	1
1.1	Problema	1
1.2	Objetivos	2
1.3	Organização do trabalho	2
2	REFERENCIAL TEÓRICO.....	3
2.1	Ordenação por inserção	3
2.2	Quicksort	4
2.2.1	<i>Análise de complexidade</i>	4
2.2.2	<i>Como o pior caso ocorre.....</i>	5
2.3	Aprimoramentos clássicos	6
2.3.1	<i>Corte para Insertion Sort.....</i>	6
2.3.2	<i>Mediana-de-três.....</i>	6
2.4	Trabalhos recentes	6
3	METODOLOGIA.....	8
3.1	Ambiente experimental	8
3.2	Organização da implementação	8
3.3	Convenções de contagem	9
3.4	Massas de teste	9
3.5	Procedimento de medição	10
3.5.1	<i>Descarte da fase de aquecimento</i>	10
3.5.2	<i>Uso da mediana.....</i>	11
3.5.3	<i>Número de repetições</i>	11
3.5.4	<i>Verificação de corretude</i>	11
3.6	Calibração empírica de M	12
4	RESULTADOS E ANÁLISE.....	13
4.1	Calibração de M	13

4.2	Comparação entre as três versões	15
4.2.1	<i>Ganho das versões híbridas</i>	18
4.2.2	<i>Efeito da mediana-de-três</i>	19
4.3	Pior caso forçado	20
4.4	Análise crítica	21
4.4.1	<i>Mais comparações, menos tempo</i>	22
4.4.2	<i>O papel das métricas determinísticas</i>	22
5	CONCLUSÃO	24
5.1	Trabalhos futuros	25
	REFERÊNCIAS	26
	APÊNDICE A – CÓDIGO-FONTE DAS IMPLEMENTAÇÕES	27
A.1	Insertion Sort	27
A.2	Partição de Hoare	27
A.3	Escolha do pivô	28
A.4	Motor único das três versões	28

1 INTRODUÇÃO

A ordenação é uma das operações mais estudadas da Ciência da Computação, tanto por sua ubiquidade prática quanto por seu papel didático na análise de algoritmos. Segundo Knuth (1998), estima-se que uma fração expressiva de todo o tempo de processamento gasto em sistemas computacionais seja dedicada a ordenar dados ou a manter dados ordenados. Ordenar não é apenas um fim em si: é a etapa que viabiliza busca binária, junções eficientes em bancos de dados, algoritmos de geometria computacional e compressão de dados.

Entre os algoritmos de ordenação interna, o *Quicksort*, proposto por Hoare (1962), ocupa posição de destaque. Seu tempo médio de execução é $O(n \log n)$ com constantes multiplicativas pequenas, e ele opera essencialmente *in place*, exigindo memória auxiliar apenas para a pilha de recursão. Por esse motivo é o algoritmo adotado, em variantes refinadas, pelas bibliotecas padrão de praticamente todas as linguagens de programação de uso geral.

O Quicksort, contudo, apresenta duas fragilidades bem conhecidas. A primeira é seu pior caso: quando o pivô escolhido é sistematicamente o menor ou o maior elemento do subvetor, as partições tornam-se maximamente desbalanceadas e o algoritmo degenera para $\Theta(n^2)$ comparações. A segunda é de natureza prática: para subvetores pequenos, o custo fixo da recursão e da partição supera o benefício da estratégia de divisão e conquista, e algoritmos assintoticamente piores — como o *Insertion Sort* — tornam-se efetivamente mais rápidos.

1.1 Problema

Este trabalho trata do seguinte problema: **quanto se ganha, na prática, ao corrigir essas duas fragilidades do Quicksort, e sob quais condições cada correção compensa?** A pergunta não é respondida apenas pela análise assintótica, uma vez que ambas as correções preservam a complexidade $O(n \log n)$ do caso médio e atuam justamente sobre as constantes multiplicativas que a notação assintótica descarta.

1.2 Objetivos

O objetivo geral é realizar um estudo comparativo experimental de três implementações do Quicksort, conforme o exercício 15 do capítulo 4 de Ziviani (2018):

1. **Quicksort recursivo:** versão clássica, com recursão até subvetores unitários;
2. **Quicksort híbrido:** versão em que a partição é interrompida para subvetores com menos de M elementos, os quais são ordenados por Insertion Sort, sendo o valor de M determinado empiricamente;
3. **Quicksort híbrido com mediana-de-três:** aprimoramento da versão anterior, em que o pivô é a mediana entre o primeiro, o central e o último elemento do subvetor.

São objetivos específicos: (i) determinar empiricamente o valor de M ; (ii) medir tempo de execução, número de comparações e número de trocas sobre massas de teste variadas; (iii) forçar explicitamente o pior caso do Quicksort e verificar experimentalmente seu comportamento quadrático; e (iv) analisar criticamente os resultados à luz da teoria, justificando as diferenças observadas entre as versões.

1.3 Organização do trabalho

O Capítulo 2 apresenta o referencial teórico, com a descrição dos algoritmos Insertion Sort e Quicksort e a análise de complexidade de ambos, com destaque para o pior caso do Quicksort. O Capítulo 3 descreve a metodologia adotada, incluindo as massas de teste, as convenções de contagem e os cuidados tomados na medição de tempo. O Capítulo 4 apresenta e analisa os resultados obtidos. O Capítulo 5 sintetiza as conclusões do trabalho.

2 REFERENCIAL TEÓRICO

Este capítulo apresenta os dois algoritmos de ordenação objeto deste trabalho — o Insertion Sort e o Quicksort — e a análise de complexidade de ambos, com ênfase nas condições que levam o Quicksort ao seu pior caso.

2.1 Ordenação por inserção

O *Insertion Sort*, ou ordenação por inserção, constrói o vetor ordenado incrementalmente. A cada iteração i , o algoritmo assume que o prefixo $v[0..i-1]$ já está ordenado e insere $v[i]$ na posição correta desse prefixo, deslocando para a direita todos os elementos maiores que ele. É o método que, segundo Ziviani (2018), corresponde à forma como um jogador ordena as cartas na mão.

Sua análise é direta. Seja $C(n)$ o número de comparações entre chaves. No **melhor caso** — vetor já ordenado — cada elemento é comparado apenas com seu predecessor imediato e nenhum deslocamento ocorre, resultando em

$$C_{\min}(n) = n - 1 = \Theta(n). \quad (2.1)$$

No **pior caso** — vetor em ordem inversa — o elemento $v[i]$ precisa ser comparado com todos os i elementos anteriores, de modo que

$$C_{\max}(n) = \sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} = \Theta(n^2). \quad (2.2)$$

No **caso médio**, supondo permutações equiprováveis, cada elemento percorre em média metade do prefixo, resultando em aproximadamente $n^2/4$ comparações, o que também é $\Theta(n^2)$ (CORMEN et al., 2022).

Apesar da complexidade quadrática, o Insertion Sort possui três propriedades que o tornam vantajoso para entradas pequenas, e que fundamentam sua utilização nas versões híbridas deste trabalho:

1. **Constantes multiplicativas muito baixas:** o laço interno consiste em uma comparação e uma atribuição, sem chamadas de função nem manipulação de pilha;
2. **Adaptatividade:** em vetores quase ordenados o desempenho aproxima-se do linear, pois o número de deslocamentos é proporcional ao número de inversões da entrada;
3. **Localidade de referência:** os acessos são sequenciais e contíguos, o que favorece a hierarquia de memória e a predição de desvios dos processadores modernos.

2.2 Quicksort

O Quicksort (HOARE, 1961, 1962) é um algoritmo de divisão e conquista. Dado um subvetor $v[esq..dir]$, escolhe-se um elemento como **pivô** e efetua-se a **partição**: rearranja-se o subvetor de modo que todos os elementos menores ou iguais ao pivô fiquem à sua esquerda e todos os maiores ou iguais, à sua direita. Em seguida, o procedimento é aplicado recursivamente às duas partes. Diferentemente do Mergesort, não há etapa de combinação: quando as chamadas recursivas retornam, o vetor já está ordenado.

Este trabalho adota o esquema de partição de Hoare (1962) na forma apresentada por Ziviani (2018), em que dois índices caminham em sentidos opostos até se cruzarem, trocando os elementos que se encontram no lado errado do pivô. O pivô é tratado como **valor**, e não como posição; como esse valor é sempre retirado do próprio subvetor, ele funciona como sentinela natural e garante a terminação dos laços internos.

2.2.1 Análise de complexidade

O custo do Quicksort é determinado pelo **balanceamento das partições**. A partição de um subvetor de tamanho n custa $\Theta(n)$ comparações, e a recorrência geral é

$$T(n) = T(k) + T(n - k - 1) + \Theta(n), \quad (2.3)$$

onde k é o tamanho da partição esquerda.

Melhor caso. Quando o pivô é sempre a mediana exata, tem-se $k \approx n/2$ e a recorrência torna-se $T(n) = 2T(n/2) + \Theta(n)$, cuja solução, pelo Teorema Mestre, é

$$T_{\min}(n) = \Theta(n \log n). \quad (2.4)$$

Caso médio. Supondo que todas as permutações da entrada sejam equiprováveis, o número esperado de comparações é

$$C_{\text{méd}}(n) \approx 2n \ln n \approx 1,386 n \log_2 n, \quad (2.5)$$

conforme demonstrado por Hoare (1962) e detalhado por Knuth (1998). O caso médio é, portanto, apenas cerca de 39% pior que o melhor caso — resultado que explica a robustez prática do algoritmo.

Pior caso. Quando o pivô é sistematicamente o menor ou o maior elemento do subvetor, tem-se $k = 0$ a cada chamada, e a recorrência degenera para $T(n) = T(n-1) + \Theta(n)$, cuja solução é

$$C_{\text{max}}(n) = \sum_{i=1}^{n-1} (n-i) = \frac{n(n-1)}{2} \approx \frac{n^2}{2} = \Theta(n^2). \quad (2.6)$$

Além do custo quadrático em tempo, o pior caso produz uma cadeia de recursão de profundidade n , elevando o consumo de memória de $O(\log n)$ para $O(n)$ e podendo provocar estouro de pilha.

2.2.2 Como o pior caso ocorre

O ponto central — e frequentemente contraintuitivo — é que o pior caso do Quick-sort **não é uma propriedade da entrada isoladamente, mas da interação entre a entrada e a regra de escolha do pivô**. Não existe entrada intrinsecamente “ruim”; existe entrada ruim *para uma dada estratégia de pivô*.

Se a estratégia adotada é “pivô = primeiro elemento”, um vetor já ordenado é o pior caso: o menor elemento é escolhido como pivô a cada chamada, produzindo partições de tamanhos 0 e $n-1$. Esse é um cenário particularmente perigoso na prática, pois vetores já ordenados ou quase ordenados são extremamente comuns em sistemas reais — exatamente a situação em que a implementação ingênua falha.

Se a estratégia é “pivô = elemento central”, o vetor ordenado passa a ser o **melhor** caso, pois o elemento central de um vetor ordenado é precisamente sua mediana. Ainda assim, é possível construir entradas adversárias que forcem o comportamento quadrático, como demonstrado por Bentley e McIlroy (1993), que apresentam um método sistemático para gerar entradas patológicas contra qualquer regra determinística de escolha de pivô.

2.3 Aprimoramentos clássicos

2.3.1 *Corte para Insertion Sort*

A ideia de interromper a recursão em subvetores pequenos e finalizar a ordenação com Insertion Sort foi proposta por Singleton (1969) e analisada em detalhe por Sedgewick (1978). A justificativa é que, embora o Insertion Sort seja $\Theta(n^2)$, sua constante multiplicativa é muito menor que a do Quicksort; abaixo de um limiar M , o termo quadrático com constante pequena é menor que o termo $n \log n$ com constante grande.

Sedgewick (1978) sugere valores de M entre 5 e 25, faixa reproduzida em Sedgewick e Wayne (2011). Ressalte-se, entretanto, que tais valores foram obtidos em hardware da década de 1970; um dos objetivos deste trabalho é justamente determinar M empiricamente na arquitetura atual.

2.3.2 *Mediana-de-três*

A escolha do pivô como mediana entre $v[esq]$, $v[meio]$ e $v[dir]$ também é atribuída a Singleton (1969). A técnica tem dois efeitos: melhora a estimativa da mediana verdadeira do subvetor, reduzindo o desbalanceamento esperado das partições, e elimina o comportamento quadrático em entradas ordenadas e inversamente ordenadas.

Sedgewick (1978) demonstra que a mediana-de-três reduz o número esperado de comparações no caso médio de $2n \ln n$ para aproximadamente $\frac{12}{7}n \ln n$, uma redução em torno de 14%. É importante observar, contudo, que a técnica **não elimina** o pior caso: ela apenas torna as entradas patológicas mais difíceis de ocorrer por acaso. Musser (1997) demonstra esse ponto e propõe o *Introsort*, que monitora a profundidade da recursão e alterna para Heapsort quando ela excede $2 \log_2 n$, garantindo $O(n \log n)$ no pior caso. Essa é a estratégia adotada pela função `std::sort` da biblioteca padrão do C++.

2.4 Trabalhos recentes

A pesquisa sobre Quicksort permanece ativa, com foco no aproveitamento das características do hardware moderno.

Wild e Nebel (2012) realizam a análise de caso médio do Quicksort de pivô duplo de Yaroslavskiy, adotado no Java 7, mostrando que ele executa cerca de 5% menos comparações que a versão de pivô único — resultado estendido por Kushagra et al. (2014) para múltiplos pivôs, que atribuem o ganho real à redução do número de falhas de cache, e não à contagem de comparações.

Edelkamp e Weiß (2019) propõem o *BlockQuicksort*, que reorganiza a partição em

blocos a fim de eliminar desvios condicionais dependentes de dados, obtendo ganhos de até 80% em relação ao `std::sort` ao evitar erros de predição de desvio. Aumüller e Bingmann (2021) simplificam a técnica usando o esquema de partição de Lomuto. Na mesma linha, Peters (2021) apresenta o *pattern-defeating quicksort* (pdqsort), que combina detecção de padrões na entrada com partição sem desvios, hoje adotado pela biblioteca padrão da linguagem Rust. Axtmann et al. (2022) estendem essas ideias para o contexto paralelo de memória compartilhada.

O ponto comum a esses trabalhos é relevante para a análise deste trabalho: em hardware moderno, **o número de comparações deixou de ser um bom preditor isolado do tempo de execução**. Efeitos de cache e de predição de desvios respondem por parcela significativa do custo real, o que será observado experimentalmente no Capítulo 4.

3 METODOLOGIA

3.1 Ambiente experimental

Os experimentos foram executados em máquina com sistema operacional Linux (*kernel* 6.17), arquitetura x86-64. O código foi escrito em C++17 e compilado com g++ 13.3 e otimização -O2, sem qualquer diretiva específica de arquitetura, de modo a manter a reprodutibilidade em outras máquinas. A medição de tempo utiliza o relógio monotônico da máquina (`std::chrono::steady_clock`), imune a ajustes do relógio de parede durante a execução.

3.2 Organização da implementação

As três versões exigidas pelo enunciado diferem entre si em apenas dois aspectos: **onde a recursão é interrompida** (parâmetro M) e **como o pivô é escolhido** (estratégia PRIMEIRO, MEIO ou MEDIANA3). Por esse motivo, optou-se por implementar um **único motor** de ordenação, parametrizado por esses dois valores; as três versões do enunciado são atalhos com parâmetros fixos, conforme o Quadro 1.

Tabela 1 – Parametrização das três versões a partir do motor único.

Versão	M	Estratégia de pivô
(a) Quicksort recursivo	1	Elemento central
(b) Quicksort híbrido	40	Elemento central
(c) Híbrido com mediana-de-três	40	Mediana de três
(d) Pivô inadequado (pior caso)	1	Primeiro elemento

Essa decisão de projeto não visa apenas à economia de código. Ela garante que as três variantes contabilizem comparações e trocas **exatamente da mesma forma**, o que é condição necessária para a validade da comparação experimental: se cada versão possuísse seu próprio laço de partição, qualquer divergência na contagem seria indistinguível de uma diferença real de desempenho. Pelo mesmo motivo, a versão (d), usada para forçar o pior

caso, é obtida apenas trocando a estratégia de pivô, sem qualquer outra alteração no algoritmo.

3.3 Convenções de contagem

Conforme exigido pelo enunciado, os algoritmos mantêm contadores de comparações e trocas. As convenções adotadas são as seguintes:

- **Comparações:** contabiliza-se toda comparação entre *chaves* do vetor (elemento contra pivô ou elemento contra elemento). Comparações entre índices, como $i \leq j$, não são contabilizadas, por constituírem controle de laço e não dependerem dos dados;
- **Trocas:** contabiliza-se toda *movimentação* de chave dentro do vetor. No Quicksort, cada troca da partição conta como uma unidade; no Insertion Sort, cada deslocamento $v[j + 1] \leftarrow v[j]$ também conta como uma unidade.

A segunda convenção merece justificativa. Contabilizar o deslocamento do Insertion Sort como troca mantém as duas grandezas na mesma unidade semântica — “quantas vezes um elemento mudou de posição” — permitindo que os dois algoritmos sejam somados nas versões híbridas sem distorção. A alternativa, contar apenas permutações de pares, subestimaria sistematicamente o trabalho realizado pelo Insertion Sort.

3.4 Massas de teste

Foram utilizadas cinco massas de teste, descritas na Tabela 2, com n igual a 1.000, 10.000, 100.000 e 500.000 elementos.

Tabela 2 – Massas de teste utilizadas.

Massa	Descrição
Aleatório	Chaves uniformemente distribuídas em $[0, 10n)$. Representa o caso médio.
Ordenado	$0, 1, \dots, n - 1$. Melhor caso para pivô central; pior caso para pivô igual ao primeiro elemento.
Inverso	$n - 1, \dots, 1, 0$. Pior caso do Insertion Sort.
Repetidos	Apenas 10 chaves distintas, sorteadas uniformemente. Avalia o tratamento de chaves iguais.
Quase ordenado	Vetor ordenado com aproximadamente 1% dos elementos trocados de posição. Representa a situação comum na prática.

Todas as massas são geradas de forma **determinística** a partir da tripla (tipo, n , semente), utilizando o gerador Mersenne Twister com semente explícita. Isso assegura duas propriedades: todas as versões do algoritmo são medidas sobre exatamente os mesmos dados, e qualquer execução pode ser reproduzida integralmente.

A massa **Repetidos** merece observação. Com apenas 10 chaves distintas em até 500.000 posições, ela estressa o comportamento do algoritmo diante de chaves iguais. O esquema de partição de Hoare adotado neste trabalho comporta-se bem nesse cenário: quando todos os elementos de um subvetor são iguais ao pivô, os dois índices avançam simetricamente e a partição resulta balanceada. Cabe registrar que o esquema alternativo de Lomuto degeneraria para $\Theta(n^2)$ nessa mesma entrada — o que ilustra como a escolha do esquema de partição, detalhe frequentemente omitido, tem consequências assintóticas.

3.5 Procedimento de medição

O enunciado exige que cada versão seja executada várias vezes sobre as mesmas massas de dados, a fim de obter médias confiáveis. Durante os testes preliminares, verificou-se que essa exigência, isoladamente, é insuficiente. Duas decisões adicionais foram necessárias e são documentadas aqui por afetarem diretamente as conclusões.

3.5.1 Descarte da fase de aquecimento

Em medições preliminares, uma **mesma carga de trabalho** registrou **16,2 ms** na primeira execução do processo e **5,3 ms** poucos segundos depois — variação de aproximadamente três vezes, causada pelo governador de frequência do processador, e não pelo

algoritmo.

O efeito não é inócuo. Como a varredura de M é percorrida em ordem crescente, os menores valores de M eram medidos enquanto o processador ainda operava em frequência reduzida. A calibração indicava, assim, $M = 100$ como valor ótimo, enquanto o número de comparações — métrica determinística e imune a esse efeito — apontava $M \approx 20$. O “ótimo” era artefato de aquecimento.

Cabe destacar que **repetir as execuções e tirar a média não corrige esse viés**, por ser ele sistemático, e não aleatório: todas as repetições de uma configuração medida cedo são igualmente penalizadas. A correção exige descartar a fase transitória. O programa, portanto, executa uma rotina de aquecimento de dois segundos antes de qualquer medição e descarta uma execução adicional por configuração.

3.5.2 *Uso da mediana*

O ruído residual de medição — preempção pelo escalonador do sistema operacional, migração entre núcleos, interferência de outros processos — só pode *acrescentar* tempo, nunca subtraí-lo. A distribuição das medidas é, portanto, assimétrica à direita, e a média é sistematicamente puxada pelos valores atípicos.

Por essa razão, a análise apresentada no Capítulo 4 utiliza a **mediana** dos tempos. Os arquivos de resultado registram mediana, média, mínimo e desvio-padrão, de modo que a dispersão das medidas permanece documentada e verificável.

3.5.3 *Número de repetições*

O número de repetições é adaptativo, pois entradas pequenas são ordenadas em dezenas de microssegundos e sofrem proporcionalmente muito mais com transientes do sistema. Nos experimentos principais utilizaram-se 10 repetições para $n \leq 10.000$, 5 para $n = 100.000$ e 3 para $n = 500.000$. Na calibração de M , onde as diferenças são mais sutis, utilizaram-se até 200 repetições para $n = 1.000$.

3.5.4 *Verificação de corretude*

Antes de qualquer medição, uma bateria de validação compara a saída das três versões com a de `std::sort` em 509 casos, incluindo vetores vazios, unitários, com todos os elementos iguais e as cinco massas em oito tamanhos distintos. Adicionalmente, **toda** execução medida tem seu resultado verificado com `std::is_sorted`: uma implementação que não ordenasse corretamente interromperia o programa, em vez de reportar um tempo de execução artificialmente baixo.

3.6 Calibração empírica de M

Para determinar M , varreu-se um conjunto de dezenove valores, $M \in \{1, 2, 3, 4, 5, 6, 8, 10, 12, 15, 20, 25, 30, 40, 50, 60, 80, 100\}$ sobre as massas Aleatório, Ordenado e Repetidos, para $n \in \{1.000; 10.000; 100.000\}$ e para as duas estratégias de pivô.

O critério de escolha adotado **não** é o mínimo absoluto do tempo. Nas proximidades do ótimo, a curva é praticamente plana, e o mínimo absoluto passa a ser determinado por ruído: em execuções preliminares ele oscilou entre 12 e 100, sem diferença real de desempenho. Adotou-se, portanto, o critério do **menor M cujo tempo permanece dentro de 2% do melhor tempo observado**, que é reprodutível entre execuções e mantém o Insertion Sort restrito a subvetores efetivamente pequenos.

4 RESULTADOS E ANÁLISE

4.1 Calibração de M

A Figura 1 apresenta o tempo mediano de execução em função de M , e a Figura 2, o número de comparações. A Tabela 3 resume os valores obtidos.

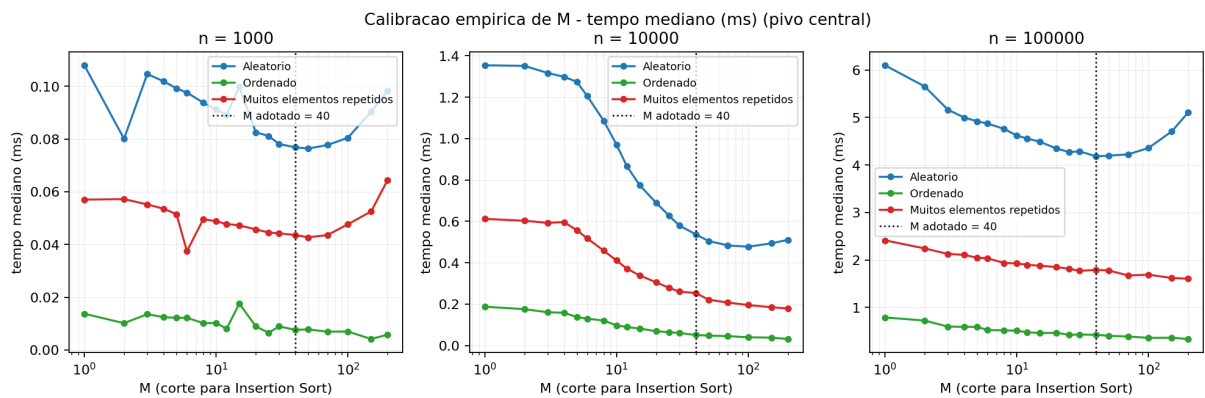


Figura 1 – Tempo mediano de execução em função de M , para três tamanhos de entrada.

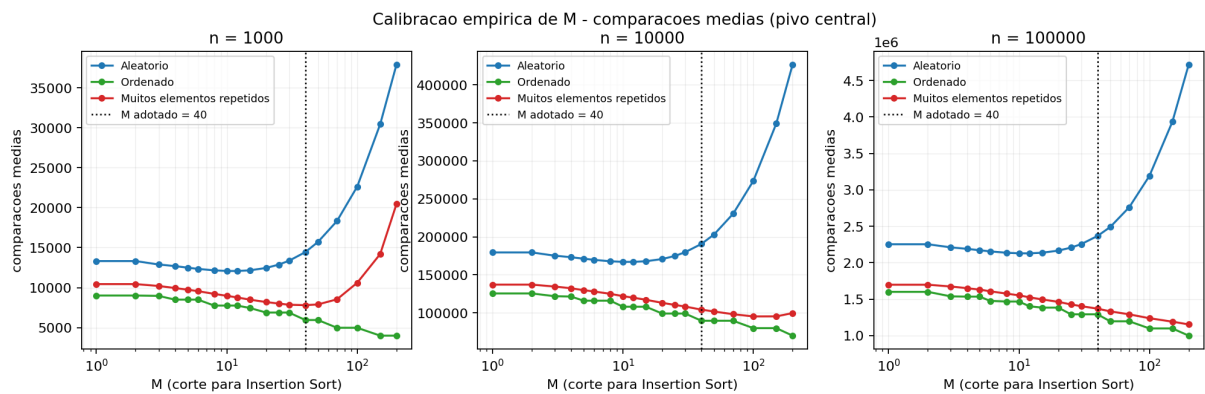


Figura 2 – Número médio de comparações em função de M .

Tabela 3 – Valores de M obtidos pela calibração empírica. A coluna “ M recomendado” usa o critério do menor M dentro de 2% do melhor tempo; a coluna “ M por comparações” usa o mínimo do número de comparações.

Estratégia de pivô	n	M recomendado	M por comparações
Elemento central	1.000	40	20
Elemento central	10.000	100	20
Elemento central	100.000	40	25
Mediana de três	1.000	40	20
Mediana de três	10.000	40	20
Mediana de três	100.000	40	25

Três observações merecem destaque.

Estabilidade. Os dois critérios comportam-se de forma distinta. O **mínimo do número de comparações** — métrica determinística, imune a ruído — é estável em $M \approx 20$ –25 nas seis células. O **mínimo de tempo**, ao contrário, variou entre 40 e 100 conforme a célula, pois o platô é plano e o *argmin* passa a ser determinado por ruído de medição. O critério do menor M dentro de 2% do melhor tempo conduz a $M = 40$ em cinco das seis células; a única exceção (pivô central, $n = 10^4$) é justamente a que evidencia a instabilidade do critério de tempo. Adotou-se $M = 40$ nos experimentos subsequentes por ser o valor recomendado na maioria das células e por manter o Insertion Sort restrito a subvetores pequenos.

Divergência entre os critérios. O mínimo do número de comparações ocorre em $M \approx 20$ –25, enquanto o mínimo do tempo ocorre em valores mais altos. A divergência é informativa: acima de $M = 25$, o algoritmo passa a executar *mais* comparações e ainda assim torna-se *mais rápido*. Esse fenômeno é analisado na Seção 4.4. O platô de tempo (valores dentro de 2% do melhor) estende-se, na maioria das células, de $M \approx 40$ a $M \approx 100$; quem fica próximo de 20 é o mínimo de **comparações**, não o de tempo.

Comparação com a literatura. O valor obtido é superior à faixa de 5 a 25 sugerida por Sedgewick (1978). A diferença é coerente com a evolução do hardware: as recomendações clássicas foram estabelecidas em máquinas nas quais o custo relativo de uma comparação era muito maior que o de um acesso à memória. Em processadores atuais, o Insertion Sort sobre blocos pequenos apresenta localidade de cache e previsibilidade de desvios excelentes, além de ser passível de vetorização pelo compilador, o que desloca o ponto de equilíbrio para valores maiores de M . Soma-se a isso o fato de as chaves utiliza-

das serem do tipo `int`, cuja movimentação é barata, favorecendo ainda mais o Insertion Sort.

4.2 Comparação entre as três versões

As Figuras 3, 4 e 5 apresentam, respectivamente, o tempo mediano, o número de comparações e o número de trocas para as três versões, em escala logarítmica nos dois eixos. As Tabelas 4 a 8 trazem os valores numéricos.

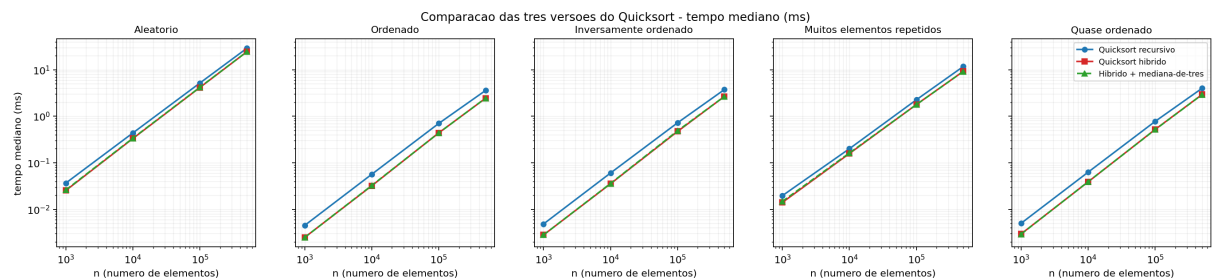


Figura 3 – Tempo mediano de execução das três versões, por massa de teste.

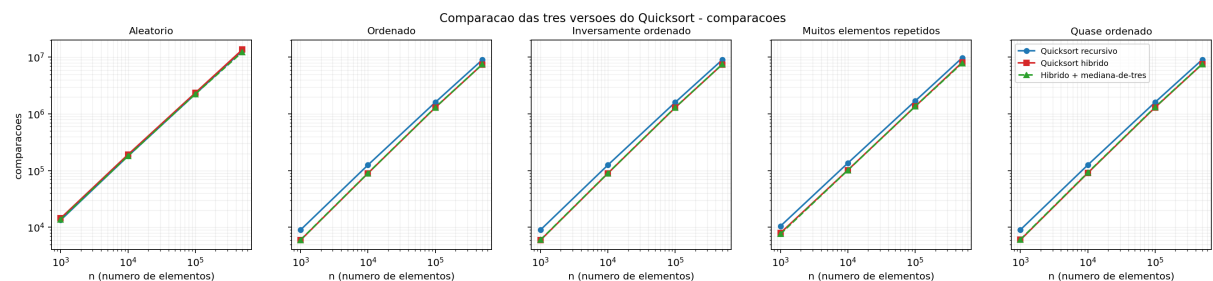


Figura 4 – Número médio de comparações das três versões, por massa de teste.

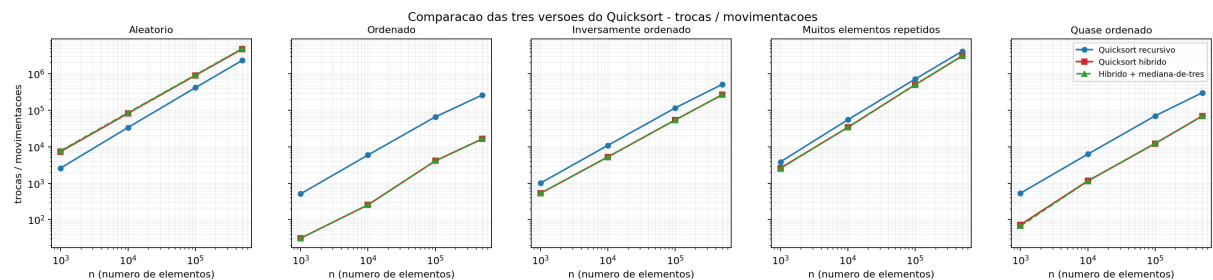


Figura 5 – Número médio de trocas (movimentações) das três versões, por massa de teste.

Tabela 4 – Resultados para a massa Aleatório.

Versão	n	Tempo (ms)	Comparações	Trocas
Recursivo	1.000	0,037	13.374	2.602
Híbrido	1.000	0,025	14.426	7.355
Mediana-3	1.000	0,026	13.846	7.736
Recursivo	10.000	0,438	180.022	33.586
Híbrido	10.000	0,332	191.362	82.044
Mediana-3	10.000	0,338	180.820	85.022
Recursivo	100.000	5,187	2.238.090	412.988
Híbrido	100.000	4,161	2.351.170	897.440
Mediana-3	100.000	4,206	2.242.550	923.525
Recursivo	500.000	28,873	13.041.400	2.332.570
Híbrido	500.000	24,262	13.610.700	4.754.780
Mediana-3	500.000	24,330	12.386.100	4.911.090

Tabela 5 – Resultados para a massa Ordenado.

Versão	n	Tempo (ms)	Comparações	Trocas
Recursivo	1.000	0,004	9.009	511
Híbrido	1.000	0,002	5.942	31
Mediana-3	1.000	0,002	6.004	31
Recursivo	10.000	0,056	125.439	5.904
Híbrido	10.000	0,032	89.497	255
Mediana-3	10.000	0,032	90.007	255
Recursivo	100.000	0,701	1.600.020	65.535
Híbrido	100.000	0,439	1.291.820	4.095
Mediana-3	100.000	0,437	1.300.010	4.095
Recursivo	500.000	3,588	9.000.020	262.143
Híbrido	500.000	2,439	7.467.250	16.383
Mediana-3	500.000	2,452	7.500.010	16.383

Tabela 6 – Resultados para a massa Inverso.

Versão	n	Tempo (ms)	Comparações	Trocas
Recursivo	1.000	0,005	9.016	1.010
Híbrido	1.000	0,003	5.946	530
Mediana-3	1.000	0,003	6.009	530
Recursivo	10.000	0,060	125.452	10.904
Híbrido	10.000	0,035	89.504	5.254
Mediana-3	10.000	0,036	90.015	5.254
Recursivo	100.000	0,723	1.600.030	115.534
Híbrido	100.000	0,471	1.291.830	54.094
Mediana-3	100.000	0,486	1.300.020	54.094
Recursivo	500.000	3,748	9.000.030	512.142
Híbrido	500.000	2,664	7.467.260	266.382
Mediana-3	500.000	2,664	7.500.030	266.382

Tabela 7 – Resultados para a massa Repetidos (10 chaves distintas).

Versão	n	Tempo (ms)	Comparações	Trocas
Recursivo	1.000	0,020	10.500	3.844
Híbrido	1.000	0,014	7.946	2.607
Mediana-3	1.000	0,015	7.633	2.536
Recursivo	10.000	0,200	135.619	54.818
Híbrido	10.000	0,157	102.243	33.791
Mediana-3	10.000	0,162	101.570	34.211
Recursivo	100.000	2,261	1.687.010	712.530
Híbrido	100.000	1,783	1.357.200	500.738
Mediana-3	100.000	1,777	1.353.860	501.874
Recursivo	500.000	11,709	9.776.490	4.131.090
Híbrido	500.000	9,215	8.124.660	3.068.950
Mediana-3	500.000	9,168	7.880.700	3.070.610

Tabela 8 – Resultados para a massa Quase ordenado.

Versão	n	Tempo (ms)	Comparações	Trocas
Recursivo	1.000	0,005	9.049	534
Híbrido	1.000	0,003	6.008	73
Mediana-3	1.000	0,003	6.099	68
Recursivo	10.000	0,063	125.767	6.252
Híbrido	10.000	0,039	90.778	1.159
Mediana-3	10.000	0,039	91.981	1.140
Recursivo	100.000	0,776	1.607.420	70.047
Híbrido	100.000	0,521	1.302.120	12.210
Mediana-3	100.000	0,516	1.316.460	12.367
Recursivo	500.000	3,988	9.043.950	300.847
Híbrido	500.000	2,927	7.527.610	69.366
Mediana-3	500.000	2,913	7.599.190	69.628

4.2.1 Ganho das versões híbridas

A Figura 6 apresenta a redução percentual de tempo das versões híbridas em relação ao Quicksort recursivo puro.

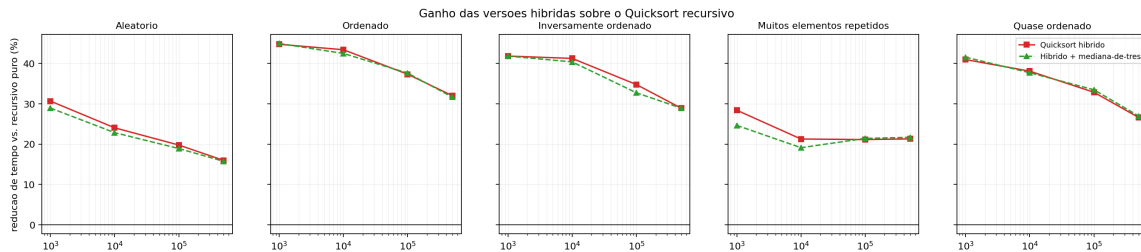


Figura 6 – Redução do tempo de execução das versões híbridas em relação ao Quicksort recursivo.

O ganho é consistente: a redução de tempo varia entre **16% e 45%**, com mediana de **31%**, sem que nenhuma configuração apresente prejuízo. Confirma-se, portanto, a hipótese que fundamenta a versão híbrida.

Cabe observar que o ganho é maior nas massas com estrutura (Ordenado, Quase ordenado) do que na massa Aleatório. A razão é a adaptatividade do Insertion Sort discutida na Seção 2.1: em subvetores quase ordenados, o número de deslocamentos é proporcional ao número de inversões, que é pequeno. Isso é visível na Tabela 5, em que o número de trocas cai de 262.143 para 16.383 em $n = 500.000$ — redução de dezesseis

vezes.

4.2.2 Efeito da mediana-de-três

A comparação entre as versões (b) e (c) exige cautela metodológica. Confrontar a versão com mediana-de-três ($M = 40$) diretamente com o Quicksort recursivo puro ($M = 1$) faria variar **duas** grandezas simultaneamente — o corte M e a estratégia de pivô — e atribuiria à mediana-de-três um ganho que decorre, em sua maior parte, do corte. Para isolar o efeito do pivô, as duas versões devem ser comparadas com o **mesmo valor de M** , conforme a Tabela 9.

Tabela 9 – Redução do número de comparações da mediana-de-três em relação ao híbrido simples, ambos com $M = 40$.

Massa	$n=1.000$	$n=10.000$	$n=100.000$	$n=500.000$
Aleatório	+4,02%	+5,51%	+4,62%	+9,00%
Repetidos	+3,94%	+0,66%	+0,25%	+3,00%
Ordenado	-1,04%	-0,57%	-0,63%	-0,44%
Inverso	-1,06%	-0,57%	-0,63%	-0,44%
Quase ordenado	-1,51%	-1,33%	-1,10%	-0,95%

Valores positivos indicam vantagem da mediana-de-três.

A Figura 7 apresenta a mesma comparação de forma visual, agora incluindo também o tempo, de modo que o efeito isolado do pivô (com M constante) fica explícito.

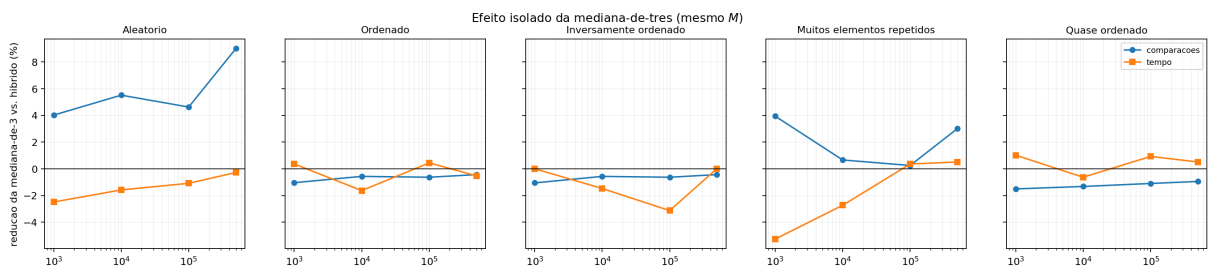


Figura 7 – Efeito isolado da mediana-de-três, ambos com $M = 40$: redução percentual das comparações e do tempo em relação ao híbrido simples. Valores positivos indicam vantagem da mediana-de-três.

O resultado é nítido e inteiramente explicado pela teoria:

- Na massa **Aleatório**, a mediana-de-três reduz as comparações em 4% a 9%. O elemento central de um vetor aleatório é um elemento qualquer da distribuição; a

mediana de três amostras é um estimador melhor da mediana verdadeira do subvetor, produzindo partições mais equilibradas. A magnitude observada é compatível com a redução teórica de aproximadamente 14% prevista por Sedgewick (1978) — a diferença decorre de o corte em $M = 40$ já eliminar as chamadas recursivas mais profundas, que são justamente aquelas em que o desbalanceamento mais pesa;

- Nas massas **Ordenado**, **Inverso** e **Quase ordenado**, a mediana-de-três é aproximadamente 1% **pior**. A explicação é direta: em um vetor ordenado, o elemento central *já é* a mediana exata do subvetor. A mediana-de-três seleciona exatamente o mesmo pivô, produzindo partições idênticas — o que se confirma pelo número de trocas rigorosamente igual nas Tabelas 5 e 6 — e gasta, além disso, duas a três comparações adicionais por chamada.

Conclui-se que, quando comparada a um pivô central, **a mediana-de-três não apresenta vantagem sistemática**. Seu valor real manifesta-se na comparação com pivôs de **posição fixa**, como se demonstra na seção seguinte.

4.3 Pior caso forçado

Para forçar explicitamente o pior caso, combinou-se **vetor já ordenado** com **escolha inadequada de pivô** (primeiro elemento), conforme discutido na Seção 2.2.2. A Tabela 10 e a Figura 8 apresentam os resultados.

Tabela 10 – Pior caso forçado: vetor ordenado com pivô igual ao primeiro elemento.

Versão	n	Tempo (ms)	Comparações	Comp./ n^2
Pivô = primeiro	1.000	0,138	501.498	0,5015
Pivô = primeiro	5.000	3,343	12.507.498	0,5003
Pivô = primeiro	10.000	13,239	50.014.998	0,5001
Pivô = primeiro	20.000	55,087	200.029.998	0,5001
Pivô = primeiro	50.000	334,355	1.250.074.998	0,5000
Recursivo (central)	50.000	0,329	750.015	0,0003
Mediana-3	50.000	0,198	600.010	0,0002

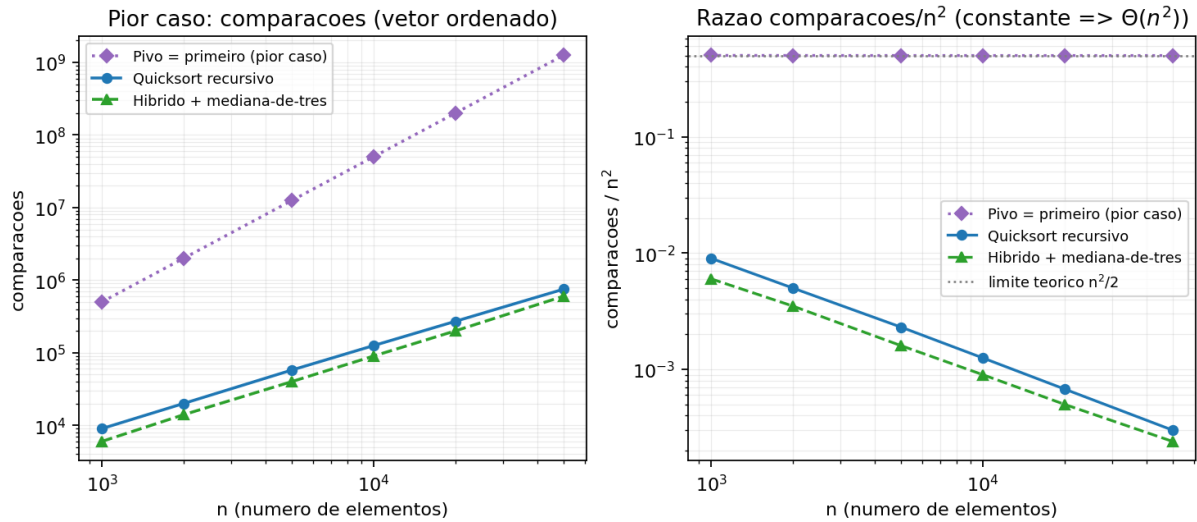


Figura 8 – Pior caso: número de comparações e razão comparações/ n^2 .

A razão comparações/ n^2 converge para **exatamente 0,5000**, confirmando numericamente a previsão teórica da Equação 2.6, segundo a qual o pior caso executa $n(n-1)/2 \approx n^2/2$ comparações. O painel direito da Figura 8 torna essa constatação visual: a curva do pivô inadequado é horizontal, sobreposta ao limite teórico, enquanto as curvas das demais versões decrescem — comportamento esperado de funções $\Theta(n \log n)$, cuja razão sobre n^2 tende a zero.

Em números absolutos, para $n = 50.000$ a versão com pivô inadequado executa 1.250.074.998 comparações contra 600.010 da versão com mediana-de-três: uma diferença de aproximadamente **2.000 vezes**, refletida em tempo de execução **1.700 vezes** maior (334,4 ms contra 0,198 ms).

Cabe registrar uma consequência prática frequentemente esquecida. No pior caso, a profundidade da recursão é n , e não $O(\log n)$. Com a pilha padrão de 8 MB do sistema utilizado, entradas superiores a aproximadamente 100.000 elementos provocariam estouro de pilha — motivo pelo qual este experimento foi limitado a $n = 50.000$. Trata-se de uma falha de natureza distinta da lentidão: o programa não apenas demora, ele termina anormalmente. Essa observação reforça o argumento de Musser (1997) em favor do monitoramento da profundidade de recursão.

4.4 Análise crítica

Dois resultados deste trabalho merecem análise mais detida, por contrariarem a intuição derivada exclusivamente da análise assintótica.

4.4.1 *Mais comparações, menos tempo*

Observe-se a massa Aleatório na Tabela 4. Para $n = 100.000$, o Quicksort recursivo executa 2.238.090 comparações em 5,187 ms, enquanto a versão híbrida executa 2.351.170 comparações — **5% a mais** — em 4,161 ms, sendo, portanto, **20% mais rápida**. O mesmo padrão repete-se em todos os tamanhos, e também no número de trocas, que mais que dobra na versão híbrida.

À primeira vista, o resultado é paradoxal: como pode um algoritmo que realiza mais operações elementares ser mais rápido? A explicação está no fato de que **nem toda operação elementar custa o mesmo**:

1. **Custo por chamada recursiva.** Com $M = 1$, o Quicksort realiza aproximadamente n chamadas recursivas, cada uma com prólogo, epílogo, manipulação de pilha e cálculo de pivô. Com $M = 40$, esse número cai cerca de 40 vezes. As comparações economizadas na recursão são substituídas por comparações do Insertion Sort, que não têm esse custo fixo associado;
2. **Localidade de referência.** O Insertion Sort opera sobre blocos contíguos de no máximo 40 elementos — 160 bytes, que cabem em três linhas de cache. Todos os acessos ocorrem em cache L1. Já as chamadas recursivas profundas do Quicksort tocam regiões dispersas do vetor;
3. **Predição de desvios.** O laço do Insertion Sort tem padrão de desvio altamente previsível, ao passo que o desvio da partição do Quicksort depende da comparação com o pivô e é, por construção, imprevisível — cada erro de predição custa entre 15 e 20 ciclos de *pipeline*.

Esse achado é coerente com a literatura recente discutida no Capítulo 2: Kushagra et al. (2014) atribuem os ganhos do Quicksort de múltiplos pivôs à redução de falhas de cache, e não à contagem de comparações, e Edelkamp e Weiß (2019) obtêm ganhos expressivos atuando exclusivamente sobre a predição de desvios, sem alterar o número de comparações. A conclusão é que **a contagem de comparações, isoladamente, deixou de ser um preditor confiável do tempo de execução em hardware moderno** — embora permaneça a métrica correta para caracterizar a complexidade assintótica.

4.4.2 *O papel das métricas determinísticas*

O segundo resultado é de natureza metodológica. Como relatado na Seção 3.5, a primeira calibração de M produziu um valor incorreto por contaminação da medição de tempo pelo governador de frequência do processador.

O erro foi detectado porque **duas métricas mediam o mesmo fenômeno com naturezas distintas**: o número de comparações é determinístico e reprodutível bit a bit, enquanto o tempo é uma grandeza ruidosa. Quando as duas discordaram sistematicamente, a métrica determinística evidenciou a existência de um problema na medição.

Esse é, precisamente, o motivo pelo qual o enunciado exige a manutenção de contadores de comparações e trocas **além** da medição de tempo. Os contadores não são apenas uma métrica complementar de desempenho: funcionam como instrumento de verificação da própria medição experimental. Sem eles, o valor incorreto de M teria sido incorporado a este relatório sem qualquer sintoma perceptível.

5 CONCLUSÃO

Este trabalho realizou um estudo comparativo experimental de três implementações do Quicksort — recursivo puro, híbrido com Insertion Sort e híbrido com mediana-de-três — avaliadas sobre cinco massas de teste e quatro tamanhos de entrada, com medição de tempo de execução, número de comparações e número de trocas.

Quanto ao **primeiro objetivo específico**, a calibração empírica mostrou que o mínimo do número de comparações — métrica determinística — é estável em $M \approx 20$ –25, enquanto o mínimo de tempo é sensível a ruído e variou entre 40 e 100. Adotou-se o critério do menor M dentro de 2% do melhor tempo, que conduz a $M = 40$ na maioria das células. Esse valor é superior à faixa de 5 a 25 recomendada por Sedgewick (1978), diferença atribuída à evolução do hardware: em processadores atuais, a localidade de cache e a previsibilidade de desvios do Insertion Sort deslocam o ponto de equilíbrio para valores maiores.

Quanto ao **segundo objetivo**, as versões híbridas reduziram o tempo de execução entre 16% e 45% em relação ao Quicksort recursivo puro, com mediana de 31%, sem prejuízo em nenhuma configuração testada. O ganho foi maior nas massas com estrutura, em razão da adaptatividade do Insertion Sort.

Quanto ao **terceiro objetivo**, o experimento de pior caso confirmou numericamente a previsão teórica: a razão comparações/ n^2 convergiu para exatamente 0,5000, correspondendo às $n(n-1)/2$ comparações previstas pela análise. Para $n = 50.000$, a diferença entre a escolha inadequada e a adequada de pivô alcançou aproximadamente 2.000 vezes em número de comparações e 1.700 vezes em tempo de execução.

Quanto ao **quarto objetivo**, a análise crítica produziu dois resultados que merecem destaque.

O primeiro é que a mediana-de-três, quando comparada ao pivô central **com o mesmo valor de M** , não apresenta vantagem sistemática: reduz as comparações em 4% a 9% apenas em dados aleatórios e é cerca de 1% pior em entradas ordenadas, inversas e quase ordenadas. O resultado é plenamente explicado pela teoria — em um vetor ordenado o elemento central já é a mediana exata do subvetor, de modo que a técnica seleciona o mesmo pivô e gasta comparações adicionais sem contrapartida. Seu valor

real reside na proteção contra pivôs de posição fixa, cenário em que a diferença é de três ordens de grandeza. Cabe destacar que esse resultado só se tornou visível porque as duas versões foram comparadas mantendo M constante; compará-las com valores distintos de M atribuiria à mediana-de-três um ganho que decorre, na verdade, do corte.

O segundo é que a versão híbrida executa *mais* comparações e *mais* trocas que o Quicksort recursivo em dados aleatórios e, ainda assim, é consistentemente mais rápida. Isso evidencia que a contagem de operações elementares, embora seja a métrica correta para a análise assintótica, deixou de ser um preditor confiável do tempo de execução em hardware moderno, no qual efeitos de cache e de predição de desvios respondem por parcela expressiva do custo real — em consonância com Kushagra et al. (2014) e Edelkamp e Weiß (2019).

Registra-se, por fim, uma contribuição de natureza metodológica. A primeira calibração realizada produziu um valor incorreto de M em virtude do aquecimento do processador, viés sistemático que não é corrigido pela simples repetição das execuções. O erro foi detectado pela discordância entre o tempo medido e o número de comparações — métrica determinística e imune ao efeito. Conclui-se que os contadores exigidos pelo enunciado desempenham papel que extrapola a caracterização de desempenho: constituem instrumento de verificação da validade da própria medição experimental.

5.1 Trabalhos futuros

Como desdobramentos naturais deste estudo, sugerem-se: a implementação do *Introsort* de Musser (1997), que garante $O(n \log n)$ no pior caso mediante monitoramento da profundidade de recursão; a avaliação do Quicksort de pivô duplo analisado por Wild e Nebel (2012); e a implementação das técnicas de partição sem desvios propostas por Edelkamp e Weiß (2019) e Peters (2021), cujo ganho, segundo os autores, decorre de fatores que a análise assintótica clássica não captura — hipótese que os resultados da Seção 4.4 deste trabalho corroboram.

REFERÊNCIAS

- AUMÜLLER, M.; BINGMANN, T. Simple and fast BlockQuicksort using Lomuto's partitioning scheme. In: PROCEEDINGS OF THE SYMPOSIUM ON ALGORITHM ENGINEERING AND EXPERIMENTS (ALENEX). [S.l.: s.n.], 2021. p. 15–26.
- AXTMANN, M. et al. Engineering in-place (shared-memory) sorting algorithms. ACM TRANSACTIONS ON PARALLEL COMPUTING, v. 9, n. 1, p. 1–62, 2022.
- BENTLEY, J. L.; MCILROY, M. D. Engineering a sort function. SOFTWARE: PRACTICE AND EXPERIENCE, v. 23, n. 11, p. 1249–1265, 1993.
- CORMEN, T. H. et al. INTRODUCTION TO ALGORITHMS. 4. ed. Cambridge, MA: MIT Press, 2022.
- EDELKAMP, S.; WEISS, A. BlockQuicksort: Avoiding branch mispredictions in Quicksort. ACM JOURNAL OF EXPERIMENTAL ALGORITHMICS, v. 24, p. 1–22, 2019.
- HOARE, C. A. R. Algorithm 64: Quicksort. COMMUNICATIONS OF THE ACM, v. 4, n. 7, p. 321, 1961.
- HOARE, C. A. R. Quicksort. THE COMPUTER JOURNAL, v. 5, n. 1, p. 10–16, 1962.
- KNUTH, D. E. THE ART OF COMPUTER PROGRAMMING, VOLUME 3: SORTING AND SEARCHING. 2. ed. Reading, MA: Addison-Wesley, 1998.
- KUSHAGRA, S. et al. Multi-pivot Quicksort: Theory and experiments. PROCEEDINGS OF THE MEETING ON ALGORITHM ENGINEERING AND EXPERIMENTS (ALENEX), p. 47–60, 2014.
- MUSSER, D. R. Introspective sorting and selection algorithms. SOFTWARE: PRACTICE AND EXPERIENCE, v. 27, n. 8, p. 983–993, 1997.
- PETERS, O. R. L. PATTERN-DEFEATING QUICKSORT. 2021. ArXiv:2106.05123.
- SEEDGEWICK, R. Implementing quicksort programs. COMMUNICATIONS OF THE ACM, v. 21, n. 10, p. 847–857, 1978.
- SEEDGEWICK, R.; WAYNE, K. ALGORITHMS. 4. ed. Upper Saddle River, NJ: Addison-Wesley, 2011.
- SINGLETON, R. C. Algorithm 347: An efficient algorithm for sorting with minimal storage. COMMUNICATIONS OF THE ACM, v. 12, n. 3, p. 185–186, 1969.
- WILD, S.; NEBEL, M. E. Average case analysis of Java 7's dual pivot Quicksort. LECTURE NOTES IN COMPUTER SCIENCE, v. 7501, p. 825–836, 2012.
- ZIVIANI, N. PROJETO DE ALGORITMOS: COM IMPLEMENTAÇÕES EM PASCAL E C. 3. ed. São Paulo: Cengage Learning, 2018. ISBN 9788522126590.

APÊNDICE A – CÓDIGO-FONTE DAS IMPLEMENTAÇÕES

O código-fonte completo, os arquivos de resultados em formato CSV e os scripts de geração dos gráficos estão disponíveis no repositório do trabalho. Reproduzem-se a seguir os trechos essenciais das implementações.

A.1 Insertion Sort

Código A.1 – Ordenação por inserção do subvetor $v[\text{esq}..\text{dir}]$.

```

1 void insertionSort(std::vector<int>& v, int esq, int dir, Contadores& c) {
2     for (int i = esq + 1; i <= dir; ++i) {
3         const int chave = v[i];
4         int j = i - 1;
5         // Desloca para a direita todos os elementos maiores que 'chave'.
6         while (j >= esq) {
7             ++c.comparacoes;
8             if (v[j] <= chave) break;
9             v[j + 1] = v[j];
10            ++c.trocas;
11            --j;
12        }
13        if (j + 1 != i) {
14            v[j + 1] = chave;
15        }
16    }
17 }
```

A.2 Partição de Hoare

Código A.2 – Partição em torno do valor do pivô. Ao final, $v[\text{esq}..j]$ contém os elementos menores ou iguais ao pivô e $v[i..\text{dir}]$, os maiores ou iguais.

```

1 static void particao(std::vector<int>& v, int esq, int dir,
2                     int& i, int& j, int pivô, Contadores& c) {
3     i = esq;
4     j = dir;
5     do {
6         while (true) { ++c.comparacoes; if (v[i] >= pivô) break; ++i; }
7         while (true) { ++c.comparacoes; if (v[j] <= pivô) break; --j; }
```

```

8         if (i <= j) {
9             std::swap(v[i], v[j]);
10            ++c.trocas;
11            ++i;
12            --j;
13        }
14    } while (i <= j);
15 }

```

A.3 Escolha do pivô

Código A.3 – Mediana de três valores, com contagem de comparações.

```

1  static int mediana3(int a, int b, int d, Contadores& c) {
2      ++c.comparacoes;
3      if (a > b) std::swap(a, b);    // garante a <= b
4      ++c.comparacoes;
5      if (b > d) {                    // a <= b e d < b: mediana e max(a, d)
6          ++c.comparacoes;
7          return (a > d) ? a : d;
8      }
9      return b;                      // a <= b <= d
10 }
11
12 static int escolherPivo(const std::vector<int>& v, int esq, int dir,
13                        EstrategiaPivo p, Contadores& c) {
14     const int meio = esq + (dir - esq) / 2;    // evita overflow de (esq+dir)/2
15     switch (p) {
16         case EstrategiaPivo::PRIMEIRO: return v[esq];
17         case EstrategiaPivo::MEIO:      return v[meio];
18         case EstrategiaPivo::MEDIANA3: return mediana3(v[esq], v[meio], v[
19             dir], c);
20     }
21     return v[meio];
22 }

```

A.4 Motor único das três versões

Código A.4 – Motor parametrizado pelo corte M e pela estratégia de pivô.

```

1  static void ordena(std::vector<int>& v, int esq, int dir,
2                    Contadores& c, int M, EstrategiaPivo p) {
3      if (esq >= dir) return;
4

```

```

5      // Interrompe a particao para subvetores com MENOS de M elementos.
6      // Com M <= 1 nenhum subvetor nao trivial e interrompido, e o algoritmo
7      // degenera na versao recursiva pura.
8      if (dir - esq + 1 < M) {
9          insertionSort(v, esq, dir, c);
10         return;
11     }
12
13     int i, j;
14     const int pivo = escolherPivo(v, esq, dir, p, c);
15     particao(v, esq, dir, i, j, pivo, c);
16     ordena(v, esq, j, c, M, p);
17     ordena(v, i, dir, c, M, p);
18 }
19
20 // As tres versoes exigidas sao atalhos com parametros fixos:
21 void quicksortRecursivo(std::vector<int>& v, Contadores& c, EstrategiaPivo
    p) {
22     quicksort(v, c, 1, p);
23 }
24 void quicksortHibrido(std::vector<int>& v, Contadores& c, int M) {
25     quicksort(v, c, M, EstrategiaPivo::MEIO);
26 }
27 void quicksortHibridoMediana3(std::vector<int>& v, Contadores& c, int M) {
28     quicksort(v, c, M, EstrategiaPivo::MEDIANA3);
29 }

```
